Déployez un modèle dans le cloud

Projet n°8 — OpenClassrooms Data Scientist

Stéphane BARRÉ

22 688
images

131
classes

1 280
features

eu-west-3
Paris RGPD

PySpark | **AWS EMR** | **MobileNetV2** | **Broadcast** | **PCA**

Référentiel d'évaluation

3 compétences évaluées — Toutes couvertes dans ce projet

C1

Identifier les outils cloud pour le Big Data

CE1 — Briques d'architecture Big Data identifiées (IAM, S3, EMR) · CE2 — Outils conformes RGPD — région eu-west-3 Paris

Slides 7–12 + Section Architecture AWS

C2

Utiliser les outils cloud pour manipuler les données

CE1 — Fichiers source ET résultats dans S3 RGPD · CE2 — Scripts exécutés sur cluster EMR cloud · CE3 — Sorties écrites directement sur S3 via write.parquet()

Slides 13–18 + Pipeline cloud Section IV

C3

Paralléliser des opérations de calcul avec PySpark

CE1 — Traitements critiques à l'échelle identifiés (slide suivante) · CE2— Données stockées ET traitées sur territoire européen · CE3 — Scripts Spark développés (binaryFile, UDF, PCA, StandardScaler) · CE4 — Toute la chaîne exécutée dans le cloud (Step COMPLETED)

Slides 19–29 + Step EMR COMPLETED

Problématique

Contexte — Start-up Fruits!

La start-up Fruits! souhaite développer une application mobile de reconnaissance de fruits.

Mission : compléter la chaîne de traitement Big Data de l'alternant en ajoutant :

- Broadcast des poids du modèle TensorFlow
- Réduction de dimension PCA en PySpark
- Respect du RGPD (serveurs européens)
- Démonstration d'une instance EMR opérationnelle

Compétences visées (référentiel)

C1
Identifier les outils cloud pour le Big Data

C2
Utiliser les outils cloud pour manipuler les données

C3
Paralléliser des opérations de calcul avec PySpark

Sommaire

I	Présentation des données & mission	→ C1·CE1, C2·CE1
II	Big Data — MapReduce, Hadoop, Spark	→ C2·CE2
III	Configuration AWS (IAM, S3, EMR, Bootstrap)	→ C1·CE1, C1·CE2 RGPD, C2·CE1-CE2
IV	Pipeline PySpark — Broadcast, UDF, PCA	→ C2·CE3, C3·CE1-CE4
V	Résultats & Conclusion	→ C3·CE4 COMPLETED

I

Présentation des données & définition de la mission

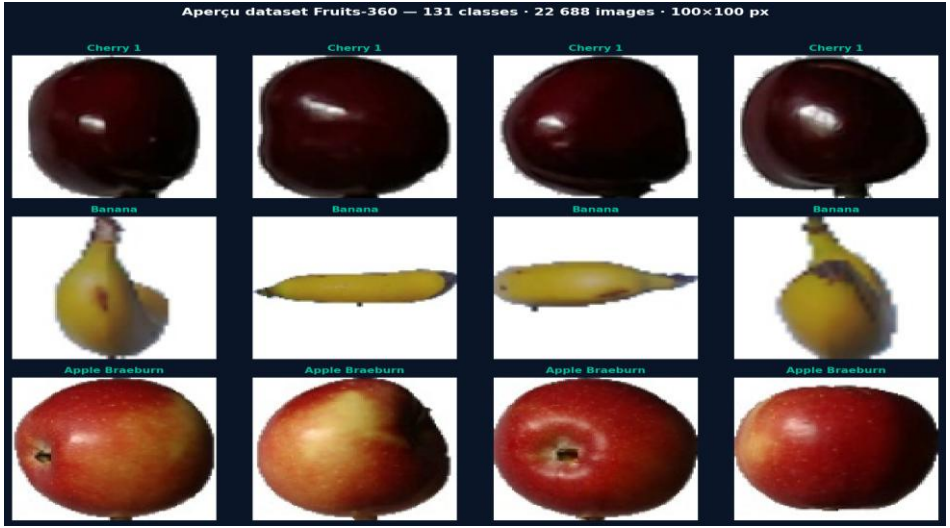
Présentation des données

Jeu de données — Fruits-360

Images totales	22 688 (Test)
Sample local	300 (validation)
Catégories	131 classes
Format	JPG — 100×100 px
Features extraites	1 280 par image
Stockage source	s3://p8-fruits-bs/Test/
Conformité RGPD	eu-west-3 Paris

Mission & Besoins techniques

Input	Images JPG depuis S3
Framework	PySpark — calcul distribué
Features	TensorFlow MobileNetV2



II

Big Data — Environnement & Outils

Qu'est-ce que le Big Data ? — Les 5V

Volume massif de données difficiles à traiter avec les outils classiques → nécessite des outils distribués

Volume	Vélocité	Variété	Véracité	Valeur
V1	**V2**	**V3**	**V4**	**V5**
22 688 images × 1 280 features = 29M valeurs — dépasse les capacités d'une seule machine	Croissance rapide attendue → scalabilité dès le départ (PySpark)	Images non-structurées (pas des tableaux classiques — binaryFile)	Qualité images : RGB, 100×100 px — format uniforme	Features extraites → classification futures des 131 classes

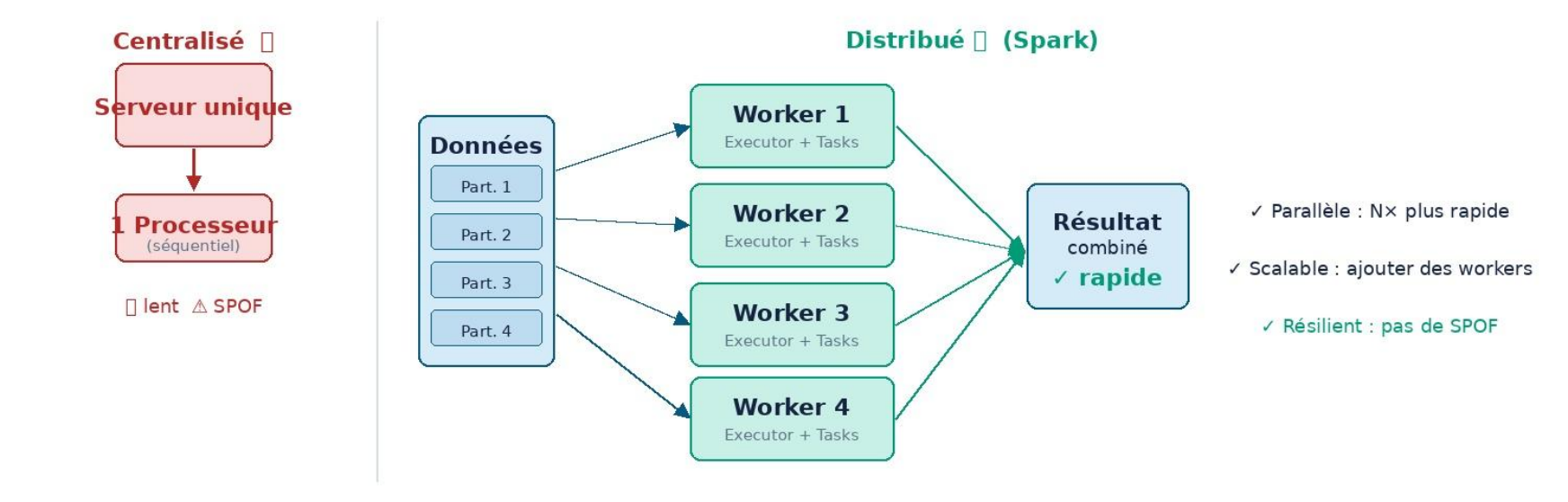
Le calcul distribué — Moteur du Big Data

Système Centralisé

- Toutes les machines dépendent d'un serveur unique
- Risque élevé de point de défaillance
- Capacité limitée à une seule machine
- Pas scalable pour le Big Data
- Si le serveur tombe → tout s'arrête

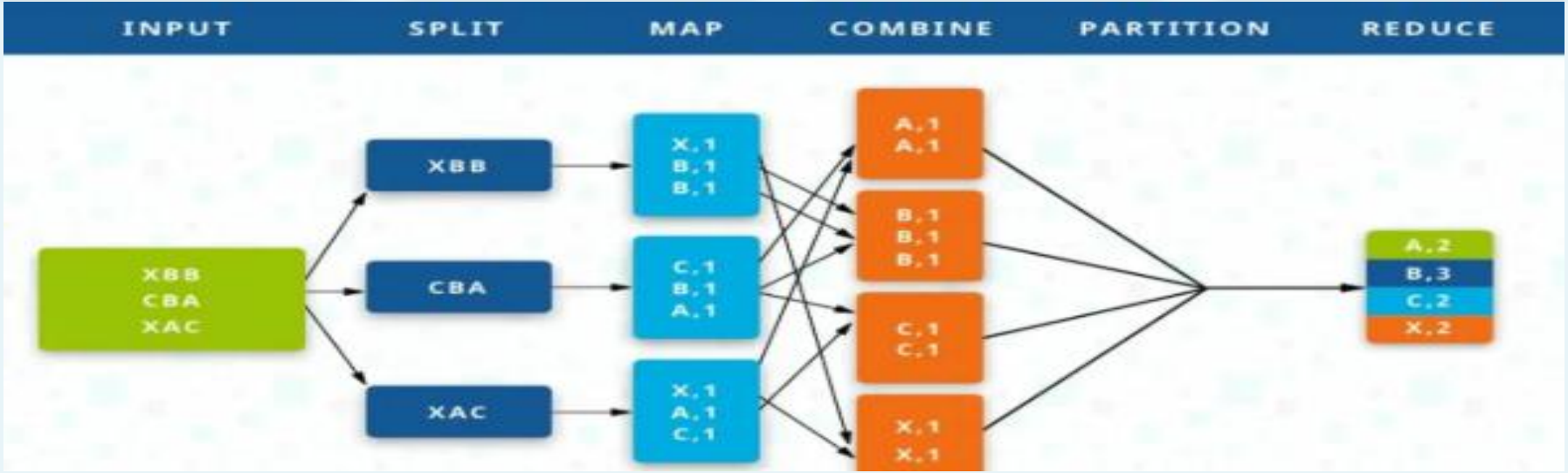
Système Distribué

- Données et calculs répartis sur plusieurs serveurs
- Résilient : une machine tombe → les autres continuent
- Scalable horizontalement : ajouter des machines
- Traitement parallèle = gain de temps × N workers
- Pattern adopté par AWS EMR pour notre projet



Hadoop & MapReduce

SPLIT	MAP	SHUFFLE	REDUCE
Découpe les données en sous-ensembles distribués	Transforme chaque élément en paire (clé, valeur)	Regroupe et trie par clé entre les workers	Agrège les résultats par clé



Composants Hadoop

- HDFS : stockage distribué tolérant aux pannes
- YARN : gestionnaire ressources (Hadoop 2.0)
- MapReduce : traitement parallèle sur le cluster
- NameNode : localisation des blocs

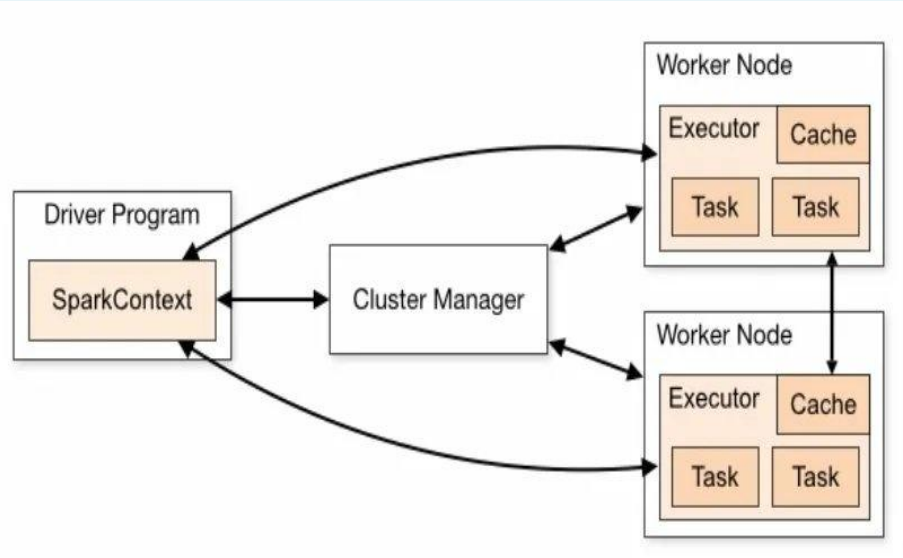
Limites → Spark

- Écriture disque entre les phases = lent
- Pas adapté au ML (algorithmes itératifs)
- Programmation complexe en Java
- → Spark : 100× plus rapide (in-memory)

Apache Spark — Standard Big Data pour le ML

Pourquoi Spark ?

- 100× plus rapide que Hadoop (calcul in-memory)
- API unifiée : SQL, ML, Streaming, Graphes
- DataFrame/Dataset : inspiré de pandas
- Compatible S3 nativement (s3://)
- PySpark : API Python native
- Lazy evaluation : plan optimisé avant exécution



Architecture Spark

Driver (SparkSession)

Orchestre le job, planifie les tâches

Cluster Manager (YARN)

Gère les ressources CPU/RAM

Executors (Workers)

Exécutent les tâches en parallèle

RDD / DataFrame

Données distribuées, lazy evaluation

Cloud & Big Data — IaaS / PaaS / SaaS

IaaS

Infrastructure as a Service

EC2 (VMs brutes)

Contrôle total

Configuration manuelle

PaaS ⍰

Platform as a Service

EMR = Spark managé

Gestion auto de l'infra

Notre choix principal

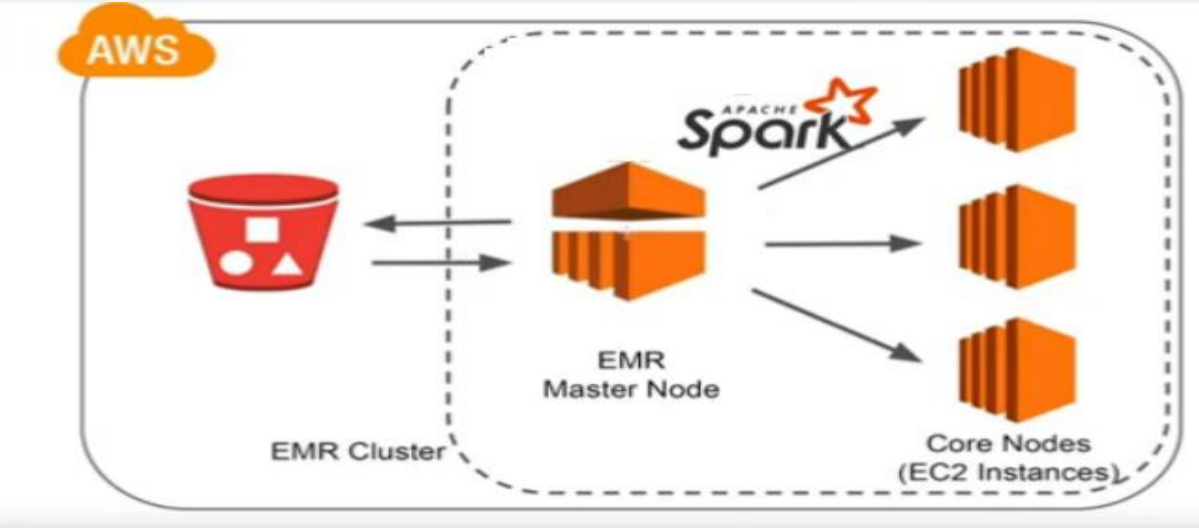
SaaS

Software as a Service

Applications prêtes

Sans configuration

Non utilisé



C1·CE2 — Conformité RGPD

Région AWS eu-west-3 (Paris) = serveurs physiquement en France. Les données ne quittent pas le territoire européen. Exigence légale pour une application grand public. Sans eu-west-3, violation du RGPD.

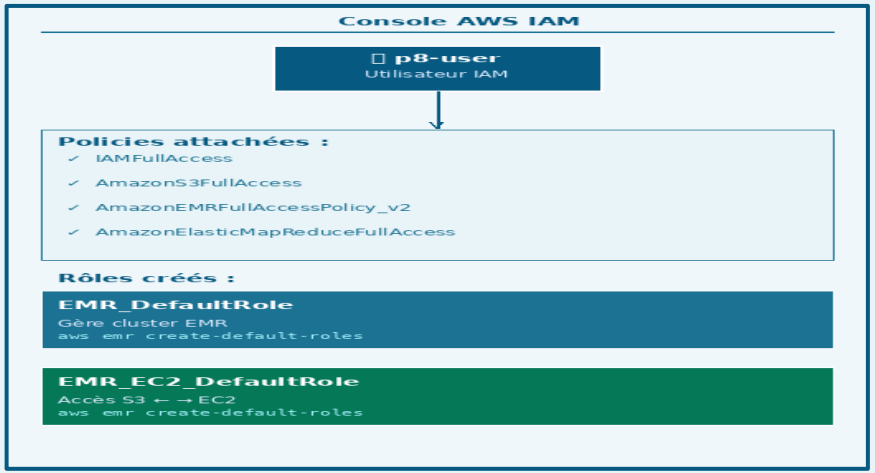
III

Configuration AWS (IAM · S3 · EMR · Bootstrap)

Amazon IAM — Gestion des droits

Utilisateur p8-user — C2·CE2

- Utilisateur IAM dédié au projet
- Policies attachées :
- • IAMFullAccess
- • AmazonS3FullAccess
- • AmazonElasticMapReduceFullAccess
- • AmazonEMRFullAccessPolicy_v2
- Credentials configurés via AWS CLI



Rôles IAM créés — C1·CE1

EMR_DefaultRole

Permet au service EMR de gérer le cluster (EC2, YARN, S3)

EMR_EC2_DefaultRole

Permet aux nœuds EC2 d'accéder à S3 : lire images + écrire résultats

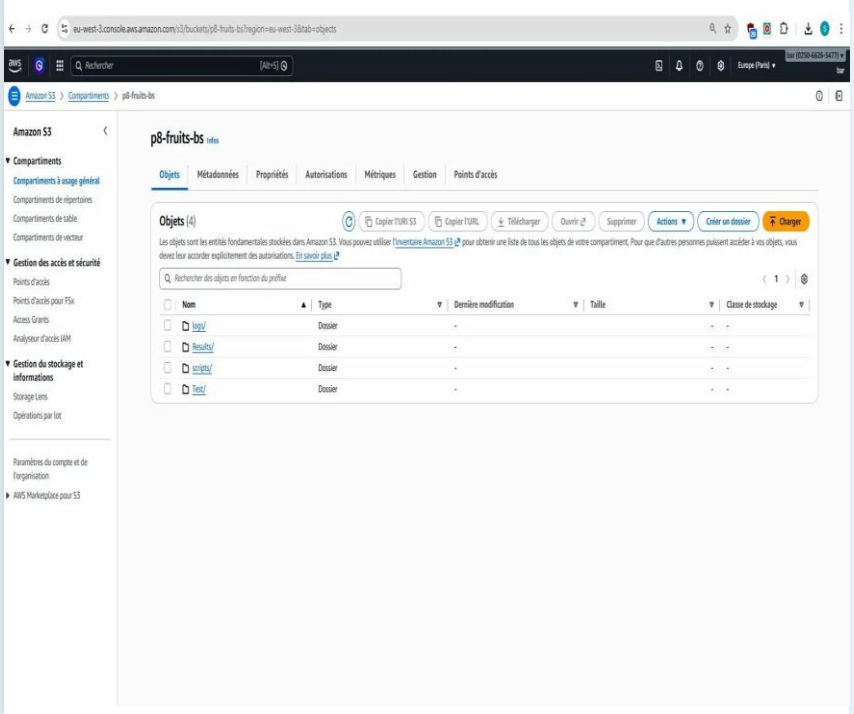
EMR_AutoScaling_DefaultRole

Permet l'autoscaling automatique du cluster

```
aws emr create-default-roles
```

Amazon S3 — Stockage Cloud RGPD

Bucket S3 — C2·CE1



Pourquoi S3 et pas HDFS ?

- S3 persiste après résiliation du cluster
- Facturation à l'usage (pas de surcoût fixe)
- Compatible nativement avec Spark (s3://)
- Chiffrement en transit et au repos
- Pas de limite de taille
- C2·CE1 : fichiers source ET résultats dans S3
- C2·CE3 : écriture directe write.parquet(s3://...)

Bootstrap Action — Installation des packages

Script shell exécuté sur CHAQUE nœud AVANT Spark — pré-requis pour C3·CE3 (scripts Spark fonctionnels dans le cloud)

```
#!/bin/bash
sudo pip3 install --upgrade pip
sudo pip3 install 'urllib3<2.0' tensorflow-cpu==2.11.0 pillow numpy pyarrow pandas
```

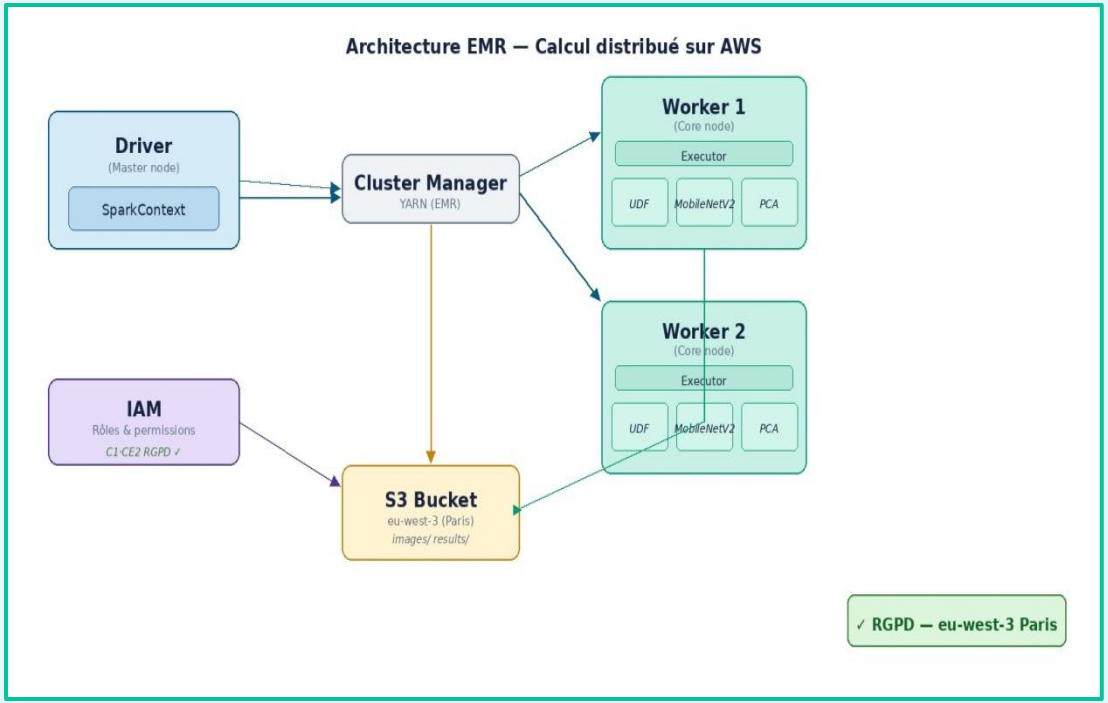
Bootstrap Action — exécuté sur chaque nœud AVANT le démarrage de Spark

sudo pip3 install 'urllib3<2.0' tensorflow-cpu==2.11.0 pillow numpy pyarrow pandas

tensorflow-cpu	pillow	pyarrow	pandas	urllib3<2.0
MobileNetV2 Transfer Learning (CPU, Python 3.7 compatible)	Traitement images PIL redimensionnement JPG	Sérialisation Arrow Pandas UDF + Parquet	DataFrames Python Pandas UDF SCALAR_ITER	Fix OpenSSL 1.0 (Amazon Linux 2)

Configuration du Cluster EMR

Version EMR	emr-6.15.0
Framework Spark	Spark 3.4.1 + Hadoop 3.3.6
Applications	Spark + Hadoop + JupyterEnterpriseGateway
Nœud Master (1×)	m5.xlarge — 4 vCPU, 16 GB RAM
Nœud Core (1×)	m5.xlarge — 4 vCPU, 16 GB RAM
Région RGPD	eu-west-3 Paris — C1·CE2
Cluster ID	j-10D17Y4KI6SBM
Step ID	s-00367443M91CBHEI65G8 — COMPLETED
Logs	s3://p8-fruits-bs/logs/ — C2·CE3



Script soumis s3://p8-fruits-bs/scripts/P8_Fruits_Cloud_v2.py → Step EMR

Cluster EMR — Instanciation & Statuts

STARTING — Provisionnement EC2

BOOTSTRAPPING — Install tensorflow,pandas...

WAITING — Cluster prêt

RUNNING — Step Spark en cours

COMPLETED — Pipeline terminé

eu-west-3.console.aws.amazon.com/emr?region=eu-west-3#/clusterDetails/j-10D17Y4KI6SBM

aws | Rechercher [Alt+S] | Europe (Paris) | bar (0250-6626-5477) bar

Amazon EMR > EMR sur EC2: Clusters > P8-Fruits-Cluster

P8-Fruits-Cluster

Mise à jour il y a moins d'une minute | Résilier | Cloner dans AWS CLI | Cloner

⚠ Le support pour cette version d'EMR prendra fin le Aug-01-2026 et celle-ci ne sera plus éligible au support technique. AWS vous recommande vivement d'exécuter vos charges de travail sur la dernière version d'Amazon EMR afin de bénéficier des mises à jour et correctifs de sécurité essentiels. Vous pouvez également utiliser le nouvel agent de mise à niveau Spark pour moderniser vos applications existantes, à partir de la version 5.40 ou ultérieure, vers la dernière version d'EMR. Pour en savoir plus, consultez Politique de support standard d'EMR et Mises à niveau Spark.

▶ Récapitulatif

Propriétés | Actions d'amorçage | Instances (Matériel) | Étapes | Applications | Configurations | Surveillance | Évènements | identifications (0)

Étapes (1) Infos

Chaque étape est une unité de travail qui contient des instructions pour manipuler les données en vue de leur traitement par un logiciel installé sur le cluster.

Troubleshoot with AI | Annuler | Cloner | Ajouter

Étapes simultanées: 1

Filtrer les étapes par statut | Rechercher des étapes | 1

ID de l'étape	Statut	Nom	Fichiers journaux	Heure de création (UTC+01:00)	Heure de début (UTC+01:00)	Temps écoulé
s-00367443M91CBHEI65G8	Completed	P8-Pipeline	controller | syslog | stderr | stdout	15 mars 2026 à 01:31	15 mars 2026 à 01:31	20 minutes, 44 secondes

Cluster ID	j-10D17Y4KI6SBM	Step ID	s-00367443M91CBHEI65G8	Région	eu-west-3 (Paris) RGPD
Images traitées	22 688	Durée bootstrap	~8 minutes	Résultats S3	s3://p8-fruits-bs/Results/

C2·CE2 — Scripts exécutés sur machines cloud | C3·CE4 — Toute la chaîne dans le cloud

IV

Déploiement sur le Cloud — Pipeline PySpark

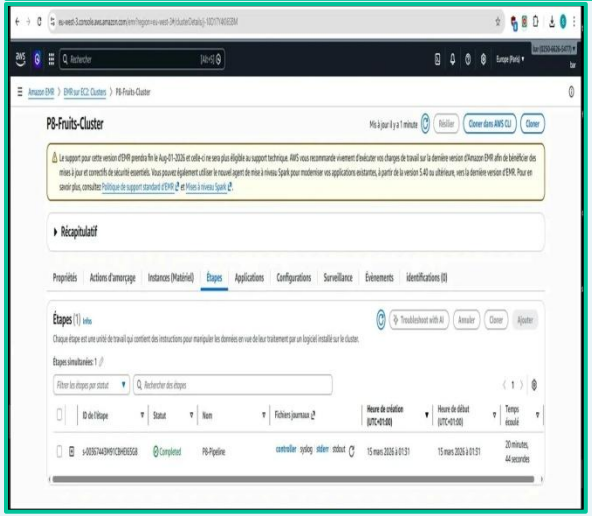
⍰ Traitements Critiques à l'Échelle— C3·CE1

Identification des opérations qui deviendraient bloquantes sans architecture distribuée (C3·CE1)

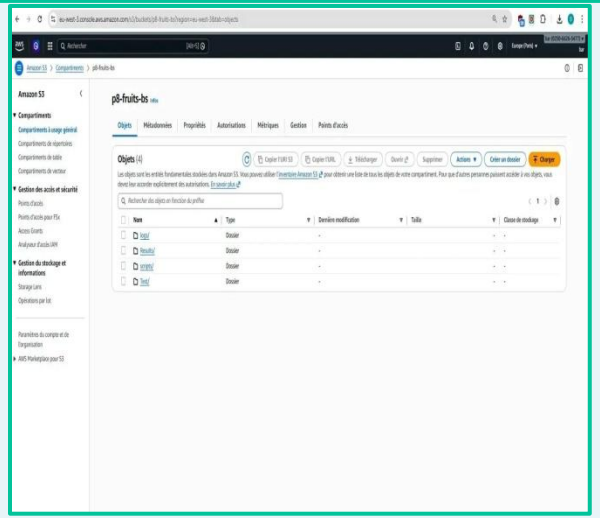
Traitement	Volume futur	Problème sans Spark	Solution retenue	CE
Chargement images	22 688 → 1M+	RAM saturée (pandas)	spark.read.binaryFile + S3	C3·CE3
Featurisation MobileNetV2	29M valeurs → 1.28 milliard	Séquentiel = plusieurs heures	Pandas UDF SCALAR_ITER	C3·CE3
Poids modèle (14 MB)	× N workers	Surcharge réseau × N	⍰ sc.broadcast()	C3·CE3
Normalisation StandardScaler	1 280 dims × 22 688	Calcul séquentiel μ et σ	StandardScaler Spark ML	C3·CE3
PCA SVD	1 280 × 1 280 matrice	Trop grande pour 1 machine	PCA Spark ML (SVD distribué)	C3·CE3
Export résultats	92 MB → 4 GB+	Écriture locale impossible	write.parquet(s3://)	C2·CE3

SparkSession & Chargement des Images

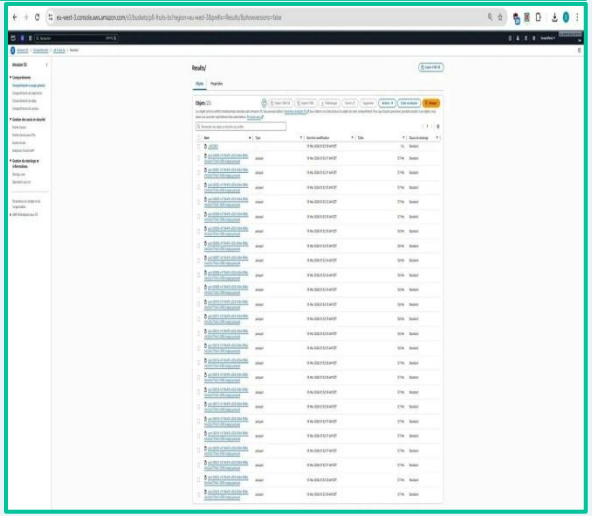

```
SparkSession & Chargement binaryFile — EMR

# Lecture des images S3 avec binaryFile
IS_EMR = os.path.exists('/mnt/var/lib/info/instance.json')

if IS_EMR:
    images = spark.read.format('binaryFile') \
        .option('pathGlobFilter', '*.jpg') \
        .option('recursiveFileLookup', 'true') \
        .load(PATH_Data)
    images = images.withColumn(
        'label',
        element_at(split(images['path'], '/'), -2)
    )
    print(f"Images chargées : {images.count()}")
    images.select('path', 'label').show(5, False)

# OUTPUT: Nombre d'images chargées : 22 688
# +----------------------------------------+-------+
# |path                                    |label  |
# +----------------------------------------+-------+
```



EMR — P8-Pipeline Completed

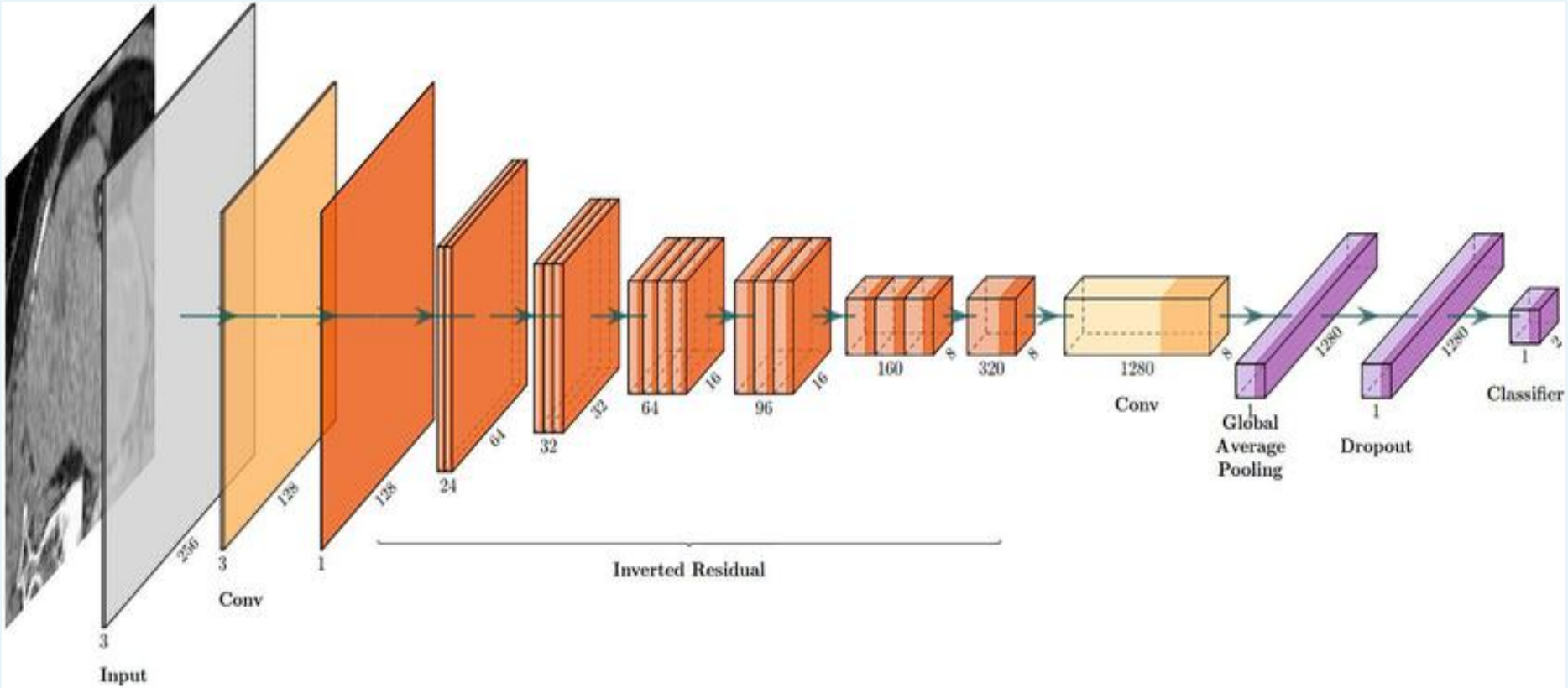


S3 — Bucket p8-fruits-bs



S3 Results/ — 25 fichiers Parquet

Transfer Learning — MobileNetV2



⍰ Broadcast des Poids— Élément Clé

sc.broadcast() — Pourquoi c'est l'étape critique ⍰

⍰ SANS broadcast

Driver
Charge model.get_weights() → 14 MB

Worker 1
↓ télécharge 14 MB depuis S3
↓ recharge le modèle en RAM

Worker 2
↓ télécharge 14 MB depuis S3
↓ recharge le modèle en RAM

Worker 3
↓ télécharge 14 MB depuis S3
↓ recharge le modèle en RAM

⋮

Worker 24
↓ 24 × 14 MB = 336 MB réseau

⍰ Coût : 24 × 14 MB = 336 MB transférés
⍰ Latence réseau × 24 workers

sc.
bcast

⍰ AVEC sc.broadcast()

Driver
brodcast_weights = sc.broadcast(model.get_weights())

1 seul envoi · 14 MB total

Worker 1 → brodcast_weights.value (RAM locale)
0 MB réseau · modèle reconstruit 1× en mémoire

Worker 2 → brodcast_weights.value (RAM locale)
0 MB réseau · modèle reconstruit 1× en mémoire

Worker 3 → brodcast_weights.value (RAM locale)
0 MB réseau · modèle reconstruit 1× en mémoire

⍰ 14 MB au lieu de 336 MB | 24× moins de réseau

Code — sc.broadcast()

```
# 1. Charger le modèle sur le DRIVER uniquement
model = MobileNetV2(weights='imagenet', include_top=True)
new_model = Model(model.input, model.layers[-2].output)

# 2. Broadcaster les poids vers TOUS les workers
brodcast_weights = sc.broadcast(new_model.get_weights())

# 3. Chaque worker reconstruit depuis la mémoire locale
def model_fn():
    m = MobileNetV2(weights=None, include_top=True)
    m.set_weights(brodcast_weights.value)  # 0 réseau
```

Pipeline PySpark — Synthèse du Code

① binaryFile + label

```
spark.read
  .format('binaryFile')
  .option('pathGlobFilter',
          '*.jpg')
  .load(PATH_Data)

images.withColumn(
  'label',
  element_at(split(
    path, '/'), -2))
```

▶ 22 688 images · 131 classes

② broadcast + UDF

```
model = MobileNetV2(
  weights='imagenet',
  include_top=True)
brodcast_weights =
  sc.broadcast(
    model.get_weights())

@pandas_udf('array<float>',
  SCALAR_ITER)
def featurize_udf(iter):
  model = model_fn()
```

▶ 1 chargement / worker □

③ repartition + Parquet

```
features_df = images
  .repartition(24)
  .select(
    col('path'),
    col('label'),
    featurize_udf(
      'content')
      .alias('features'))
features_df.write
  .parquet(PATH_Result)
```

▶ 24 partitions · s3://Results

④ Scaler + PCA

```
scaler = StandardScaler(
  withStd=True,
  withMean=True)
scaled = scaler.fit(df)
          .transform(df)

pca = PCA(k=50,
  inputCol='features_scaled')
result = pca.fit(scaled)
              .transform(scaled)
```

▶ 83.7% variance · 92 MB □

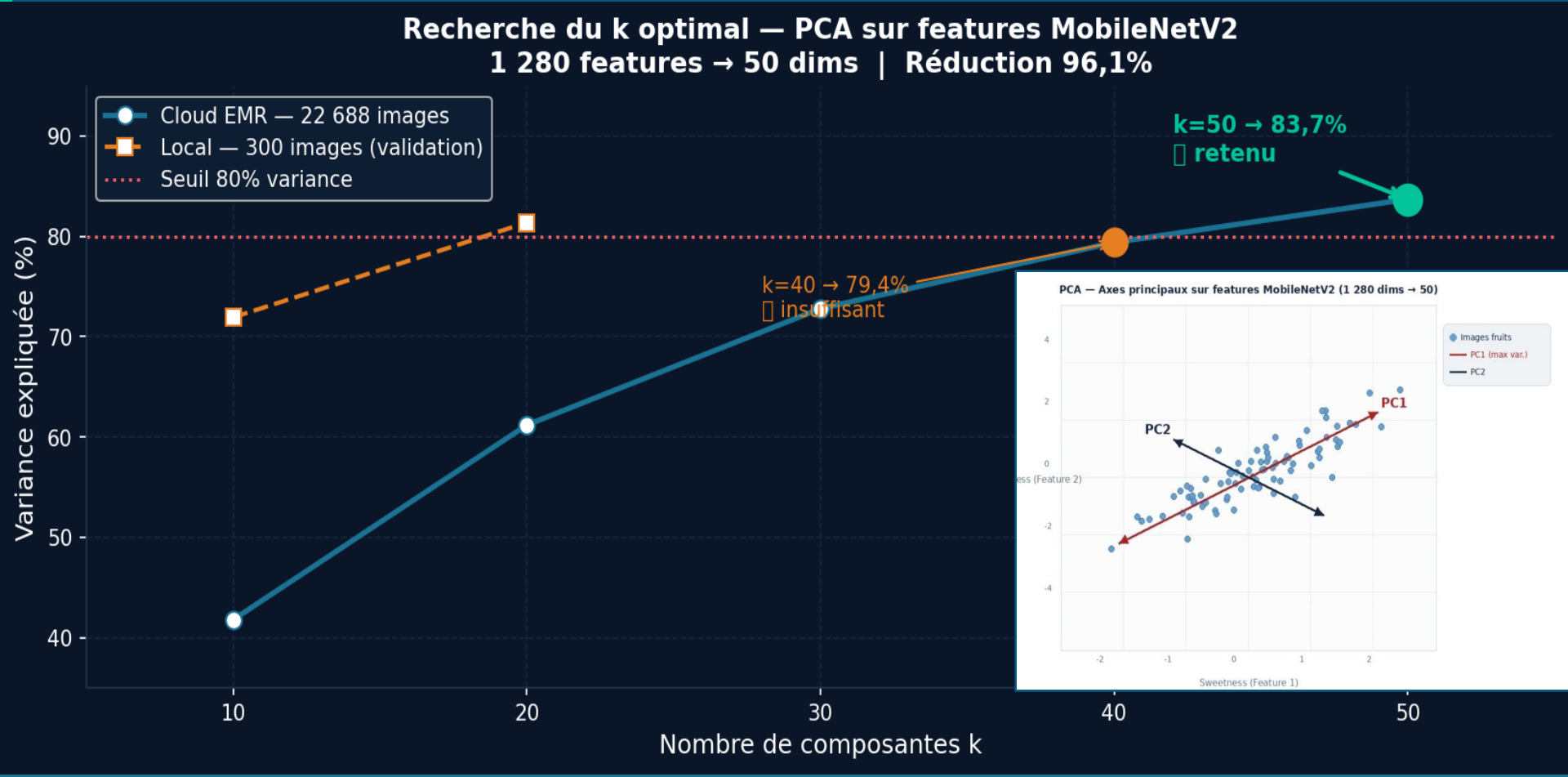
binaryFile → sc.broadcast □ → Pandas UDF SCALAR_ITER → repartition(24) → write.parquet(s3://) | 22 688 images · 1280 features · k=50 · 83.7% · 92 MB

Extraction Features — Application Cluster

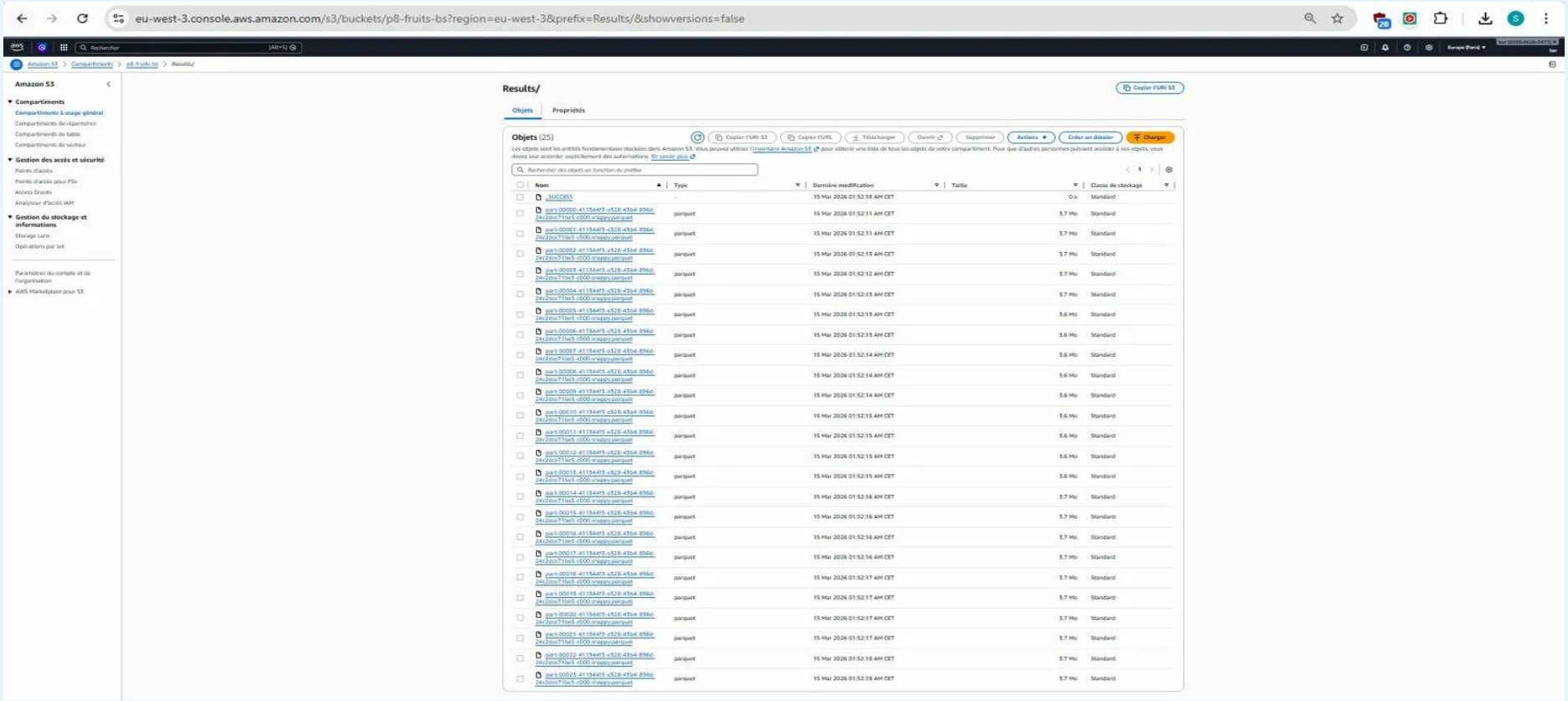
Extraction Features — Cluster EMR repartition(24)

```
# Extraction distribuée sur 24 partitions
if IS_EMR:
    features_df = images.repartition(24).select(
        col('path'),
        col('label'),
        featurize_udf('content').alias('features')
    )
    # Écriture directe S3 → déclenche le calcul
    features_df.write.mode('overwrite') \
        .parquet(PATH_Result)
    print(f'Features exportées : {PATH_Result}')

# [Stage 0:===================>] (24/24, 22688 it)
# Step s-00367443M91CBHEI65G8 — COMPLETED □
# Features exportées : s3://p8-fruits-bs/Results/
```

PCA — Recherche du k Optimal



Export Parquet → S3 & Validation



V

Résultats & Conclusion

Résultats — Pipeline Cloud Exécuté

V / Toutes compétences ⍰

22 688
images traitées
131 classes — C2·CE1 ⍰

1 280
features MobileNetV2
C3·CE3 — Script Spark ⍰

Pipeline PySpark P8 — 7 etapes

22 688 images | 1280 features MobileNetV2 | 50 dims PCA | 92MB Parquet | Step COMPLETED j-10D17Y4KI6SBM | eu-west-3 Paris RGPD

50
composantes PCA
83.7% variance — C3·CE3 ⍰

92 MB
Parquet sur S3
C2·CE3 — Sorties dans cloud ⍰

⍰ C3·CE4 — Step COMPLETED j-10D17Y4KI6SBM | C2·CE2 — Exécuté sur machines cloud | eu-west-3 Paris RGPD

Résultats : s3://p8-fruits-bs/Results/ — Parquet Snappy — Toutes compétences validées

Synthèse — Référentiel d'évaluation

✓	C1·CE1 — Architecture Big Data identifiée	IAM + S3 + EMR + Spark — tableau des rôles et fonctions
✓	C1·CE2 — Conformité RGPD	Région eu-west-3 Paris — serveurs en France — données sur territoire européen
✓	C2·CE1 — Fichiers source et résultats dans S3	s3://p8-fruits-bs/Test/ (images) + s3://p8-fruits-bs/Results/ (Parquet 92 MB)
✓	C2·CE2 — Scripts exécutés sur machines cloud	Step EMR COMPLETED sur cluster j-10D17Y4KI6SBM — eu-west-3
✓	C2·CE3 — Sorties écrites directement dans le cloud	write.parquet('s3://...') — écriture directe S3 sans passer par local
✓	C3·CE1 — Traitements critiques identifiés	Featurisation (Pandas UDF), Broadcast poids, PCA SVD, export Parquet
✓	C3·CE2 — Exploitation conforme RGPD	Données stockées ET traitées sur territoire européen (eu-west-3 Paris)
✓	C3·CE3 — Scripts Spark développés	binaryFile, repartition, pandas_udf SCALAR_ITER, StandardScaler, PCA, write.parquet
✓	C3·CE4 — Toute la chaîne dans le cloud	Step EMR COMPLETED — pipeline de bout en bout sur cluster cloud

La Suite & Conclusion

La suite du projet

- Entraîner un classifieur sur les 50 features PCA :
- • Random Forest
- • LightGBM / XGBoost
- • SVM

- Améliorer les performances :
- • Augmenter le cluster (nœuds Task)
- • Tester plus de composantes PCA

- Visualisation :
- • t-SNE 2D/3D pour explorer les clusters

Limites & Enseignements

- Limites :
- • 2 nœuds m5.xlarge = contrainte RAM PCA
- • tensorflow-cpu==2.11.0 (Python 3.7)

- Enseignements :
- • broadcast = CRITIQUE pour la performance
- • SCALAR_ITER = UDF optimal deep learning
- • Bootstrap = préparation indispensable
- • RGPD = choisir la région dès le départ

22 688 images · MobileNetV2 · Broadcast · Pandas UDF · PCA k=50 (83.7%) · 92 MB Parquet S3 · eu-west-3 Paris RGPD

Merci de votre attention

Questions ?

22 688	1 280	50	83.7%	92 MB
images	features	PCA dims	variance	Parquet S3